ModelOps как надмножество MLOps

Governance, Lifecycle Management, Business Alignment



Почему это важно? Статистика провалов

23%

Ожидалось

Gartner (2019)

5%

Реально в проде

Только моделей

84%

Считают AI критичным

Руководители

76%

Не могут масштабировать

Компании

Проблемы:

- Отсутствие governance
- Нет версионирования
- Невозможность аудита
- Сложность масштабирования

> «84% руководителей считают AI критичным, но 76% не могут его масштабировать»

От MLOps к ModelOps: эволюция

MLOps

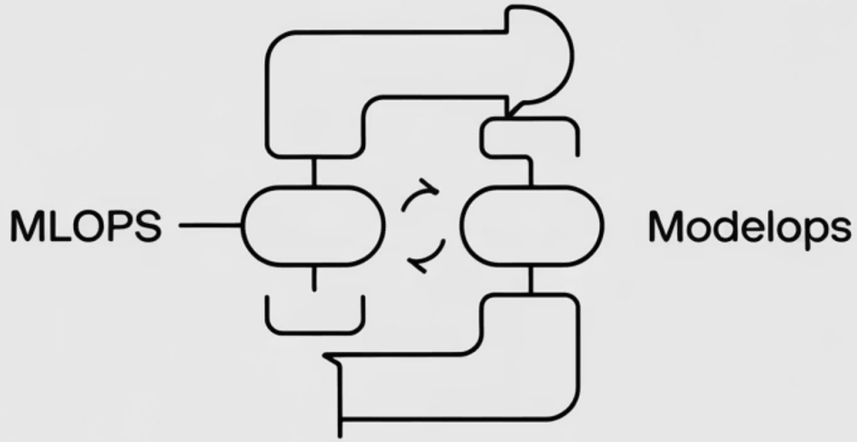
автоматизация CI/CD для ML-моделей

- ML-модели

ModelOps

MLOps + Governance + Business Alignment + All AI Models

- ML + Knowledge Graphs
- Rules + NLP + Agents



Определение ModelOps (Gartner, Neptune.ai)

> «ModelOps — это набор возможностей, сфокусированных на управлении жизненным циклом всех AI и decision-моделей с акцентом на governance»

Подчеркиваем: не только обучение, но и compliance, explainability, ротация, архивация

Что включает ModelOps? (4 pillars)

Governance

контроль, аудит, compliance

Lifecycle Management

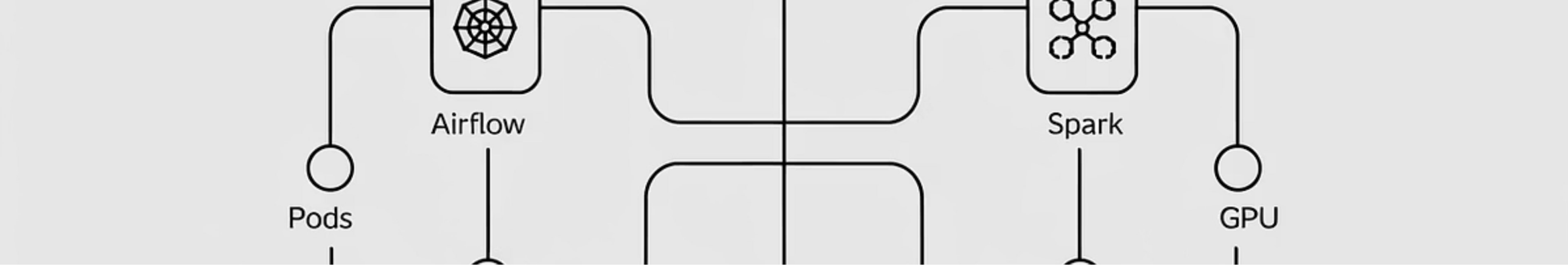
от идеи до архивации

Collaboration

роли, процессы, BPMN

Monitoring & Retraining

drift, fairness, автоматизация



Почему Kubernetes важен уже на этапе сбора данных?

Нагрузка ≠ статична:

1	2	3
Сбор данных	**Обучение**	**Инференс**
batch-джобы (Airflow + Spark on K8s)	GPU-ноды, autoscaling	low-latency, HPA

Пример: если данные приходят раз в сутки → запуск Airflow DAG → запуск Kubernetes Job → обучение → деплой

Управление ресурсами в Kubernetes под задачи ModelOps



ResourceQuotas, LimitRanges

Ограничение потребления ресурсов



Horizontal Pod Autoscaler (HPA)

Для inference



Cluster Autoscaler

Если есть



Jobs / CronJobs

Для retraining



Namespace-per-team

Для изоляции

ResourceQuotas

Что это: Механизм Kubernetes, который ограничивает суммарное потребление ресурсов (CPU, память, количество подов и т.д.) в рамках одного Namespace.

Зачем в ModelOps:

- В enterprise-среде одновременно может работать **много команд**, каждая со своими моделями.
- Без ограничений одна "жадная" модель (например, тяжёлый LLM или AutoML-эксперимент) может **заблокировать ресурсы для всех остальных**.
- ResourceQuota гарантирует **справедливое распределение** ресурсов между проектами.

Пример

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-a-quota
spec:
  hard:
    requests.cpu: "8"
    requests.memory: 16Gi
    limits.cpu: "16"
    limits.memory: 32Gi
    pods: "20"
```

→ Команда A не сможет использовать больше 16 ядер CPU и 32 ГБ RAM в своём namespace.

LimitRanges

Что это: Правила, задающие минимальные и максимальные значения ресурсов для отдельных Pod'ов или контейнеров внутри Namespace.

Зачем в ModelOps:

- Предотвращает запуск "бесконтрольных" подов без указания requests/limits.
- Защищает от ситуаций, когда **один inference-сервис "съедает" всю ноду**.
- Обеспечивает **предсказуемость** — если все поды имеют лимиты, кластер работает стабильно.

Пример

yaml

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: model-limits
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: "1Gi"
    defaultRequest:
      cpu: "200m"
      memory: "512Mi"
    max:
      cpu: "4"
      memory: "8Gi"
```

→ Любой контейнер с моделью будет автоматически получать лимиты, если их не задали явно.

3. Horizontal Pod Autoscaler (HPA) для inference

Что это: Автоматическое масштабирование количества реплик пода на основе метрик (например, CPU, память или custom-метрики вроде RPS или latency).

Зачем в ModelOps:

- **Inference-нагрузка неравномерна**: утром — пик, ночью — почти ноль.
- HPA позволяет **динамически масштабировать inference-сервисы**, чтобы:
  - не платить за простой (в on-prem — не тратить ресурсы),
  - избежать перегрузки при всплеске запросов.
- Особенно важно для **моделей, встроенных в бизнес-процессы** (например, скоринг в банке).

Как работает:

```bash
kubectl autoscale deployment model-inference --cpu-percent=70 --min=2 --max=10
```

→ Если средняя загрузка CPU > 70% — Kubernetes запустит ещё поды.

💡 В продвинутых сценариях можно использовать **custom metrics** через Prometheus Adapter (например, requests_per_second).

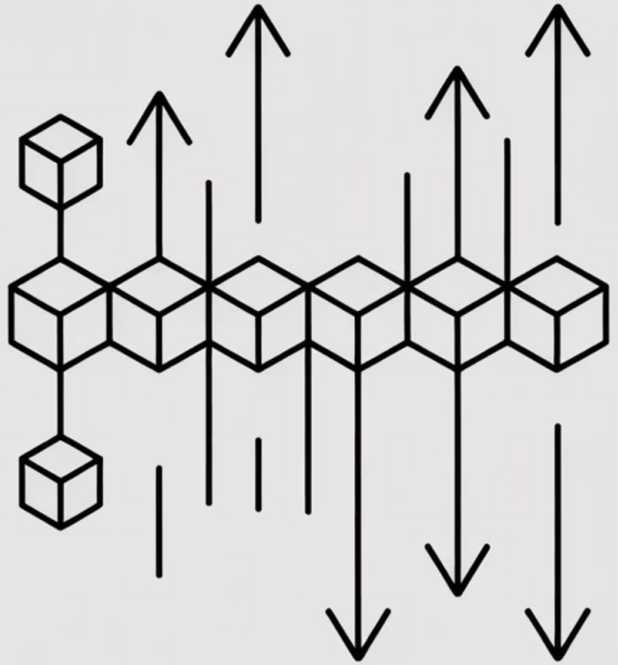
Cluster Autoscaler (если есть)

Что это: Инструмент, который автоматически добавляет или удаляет ноды в кластере, если текущих ресурсов недостаточно (или они простаивают).

Зачем в ModelOps:

- При массовом **retraining'е сотен моделей** (например, ночью) может не хватить CPU/GPU.
- Cluster Autoscaler **расширяет кластер**, чтобы уместить все Jobs.
- Утром, когда нагрузка падает — **сжимает кластер**, экономя ресурсы.

> ⚠ **Важно:** Вы сказали, что **облака нет**, но если у вас on-prem кластер на bare metal или виртуалках с поддержкой автоматического масштабирования (например, через VK Cloud, Yandex Managed Kubernetes или Rancher), то Cluster Autoscaler применим. Если нет — этот пункт можно опустить или заменить на **ручное планирование capacity**.



Cluster Autoscaling

Jobs / CronJobs для retraining

Что это:

- **Job** — однократная задача (например, обучение модели).
- **CronJob** — периодическая задача по расписанию (например, ежедневный retrain).

Пример CronJob для ежедневного retrain'a:

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: daily-retrain
spec:
  schedule: "0 2 * * *"  # каждый день в 2:00
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: trainer
            image: my-ml-trainer:v1
            command: ["python", "train.py"]
          restartPolicy: OnFailure
```

Зачем в ModelOps:

- ModelOps требует **автоматического жизненного цикла**:
  - при обнаружении **data drift** → запустить retraining → зарегистрировать новую версию → протестировать → деплоить.
- Kubernetes **Jobs** идеально подходят для этого:
  - изолированы,
  - имеют статус (Success/Failed),
  - логируются,
  - могут запускаться из Airflow или по событию мониторинга.

Namespace-per-team для изоляции

Что это: Каждая команда (или проект) работает в отдельном Kubernetes Namespace.

Зачем в ModelOps:

- **Безопасность и изоляция**: одна команда не видит поды/сервисы другой.
- **Упрощённое управление**: квоты, RBAC, сетевые политики — на уровне namespace.
- **Чистота в production**: легко понять, какая модель к какой команде относится.
- Поддержка **multi-tenancy** — критично в enterprise.

Пример:

fraud-detection-team	recommendation-team	llm-assistant-team

→ Каждый namespace может иметь свои ResourceQuota, NetworkPolicy, RoleBindings.

MLOps vs ModelOps ключевые различия

Критерий	MLOps	ModelOps
Объект управления	ML-модели	Все AI-модели
Фокус	Автоматизация пайплайнов	Governance + бизнес-согласование
Ответственность	DS + DevOps	DS + Validator + Compliance + Business Owner
Инструменты	MLflow, Kubeflow	SAS Model Manager, Datatron, open-source аналоги

Governance

Не просто «контроль» — это доверие к модели

Требования:

Воспроизводимость

Возможность повторить результат

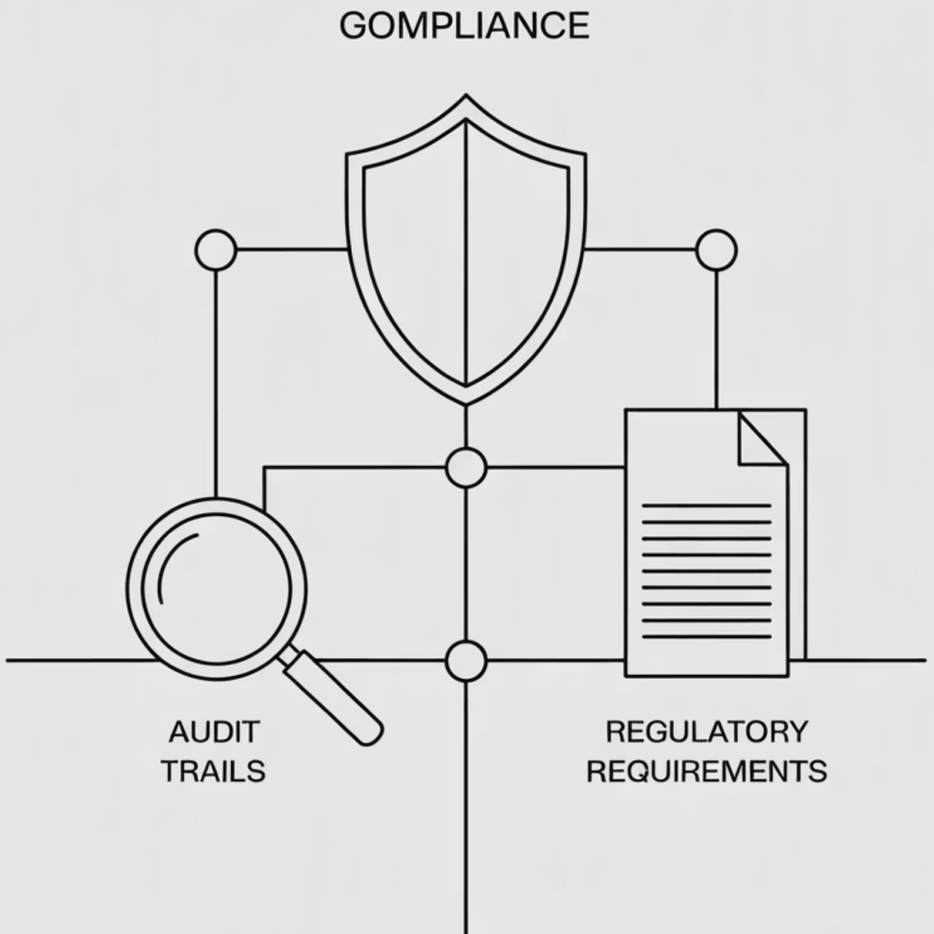
Аудит изменений

Полная история модификаций

Соответствие регуляторным нормам

например, ЦБ РФ, GDPR

Пример: кредитный скоринг → модель должна быть объяснима и справедлива



Версионирование моделей

Что версионируем:



Код модели

Алгоритмы и логика



Данные (DVC)

Обучающие датасеты



Гиперпараметры

Конфигурация модели



Метрики

Показатели качества



Окружение

Dockerfile / Poetry.lock

Инструменты: MLflow Model Registry, DVC, Git LFS

Lineage: от данных до решения

1

Data

Исходные данные

2

Features

Признаки

3

Model

Обученная модель

4

Prediction

Прогноз

5

Business Decision

Бизнес-решение

Важно для:

- Отладки
- Аудита
- Понимания причин дрифта

Open-source решение: MLflow + Evidently + custom metadata DB

Репозиторий моделей: центр истины

Централизованное хранилище:

Статус

Staging / Production / Archived

Владелец

Ответственный за модель

Дата последнего retrain

Актуальность модели

Ссылка на документацию

Model Cards и описание

Аналог SAS Model Manager → MLflow UI + PostgreSQL backend



Роли в ModelOps (по SAS и Gartner)



Data Scientist

разработка, обучение



Validator

проверка fairness, bias, метрик



DevOps / MLOps Engineer

деплой, мониторинг



Business Owner

утверждение, KPI



Compliance Officer

регуляторные проверки

BPMN: как согласовать модель с бизнесом?

Business Process Model and Notation

01

Разработка

Создание модели

02

Валидация

Проверка качества

03

Утверждение

Бизнес-одобрение

04

Деплой

Внедрение в продакшн

05

Мониторинг

Отслеживание работы

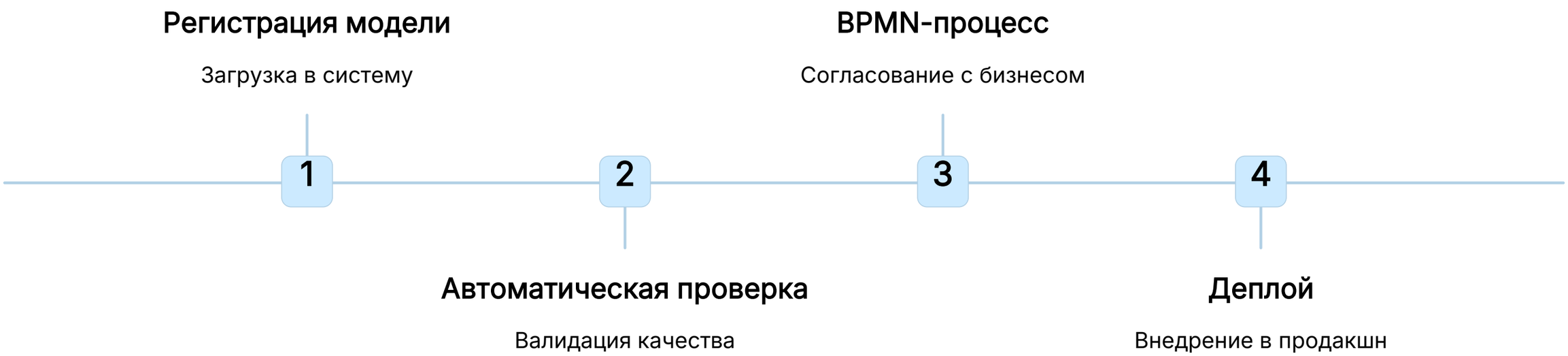
ĆĎ

Retrain

Переобучение

Human-in-the-loop: ручное подтверждение перед продом

Case Study: SAS Model Manager + Workflow Manager



Интеграция с:

- Jupyter, Git, CI/CD

Governance:

- полный аудит, Model Cards, SLA

Model Cards: документация для доверия

Что включает:



Цель модели

Бизнес-задача



Метрики

Показатели качества



Ограничения

Границы применимости



Bias/Fairness анализ

Справедливость модели



Версия данных

Используемые датасеты

Пример: Google Model Card Toolkit → open-source шаблон в Markdown

Explainability for Compliance

Почему нужно:

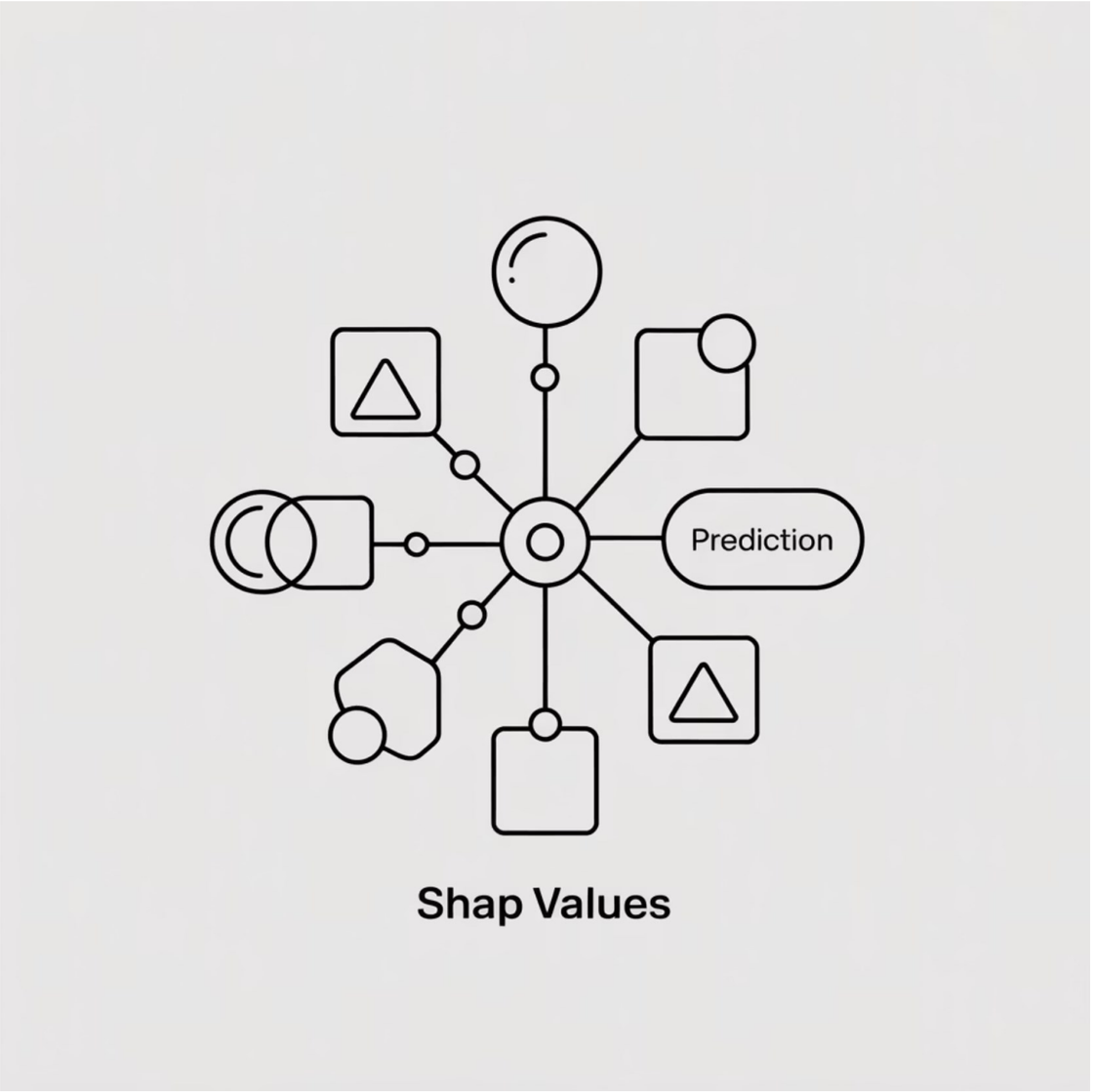
- Регуляторы требуют объяснимости
- Бизнес должен понимать решения

Инструменты:

- SHAP, LIME, Counterfactuals

Интеграция в API:

- /explain endpoint



Жизненный цикл модели: 6 этапов

Design

бизнес-требования

Develop

обучение, эксперименты



Validate

fairness, метрики, compliance

Deploy

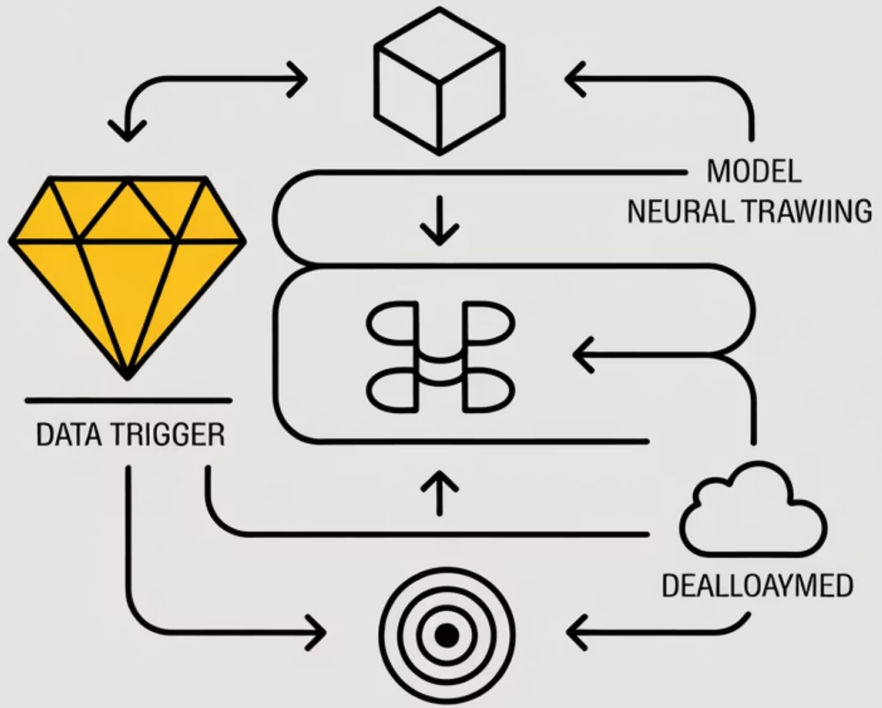
в Kubernetes, с мониторингом

Monitor

drift, decay, SLA

Retire/Archive

когда модель устарела



Автоматизация жизненного цикла

Триггеры:

1. **Data drift**
   → retrain
2. **Падение accuracy**
   → fallback
3. **Ручное утверждение**
   → деплой

Оркестрация: Airflow DAG с Manual Trigger + Slack-уведомления

Пример: Credit Scoring в банке

Требования:

- Объяснимость
- Fairness по полу/возрасту
- Аудит всех решений

ModelOps-решение:

- Model Card + SHAP
- BPMN-процесс утверждения
- Мониторинг дрифта через Evidently



Open-source стек для ModelOps

Регистрация MLflow Model Registry	**Оркестрация** Airflow + Kubernetes
Мониторинг Evidently + Prometheus	**Документация** Model Cards (Markdown)
Explainability SHAP/LIME	**Governance** Git + PostgreSQL + RBAC в K8s

Как заменить SAS в open-source мире?

SAS компонент	Open-source аналог
SAS Model Manager	MLflow + Airflow + Evidently
Workflow Manager	Airflow BPMN-подобные DAGs
Compliance Reporting	Jupyter + PDF-генерация (WeasyPrint)

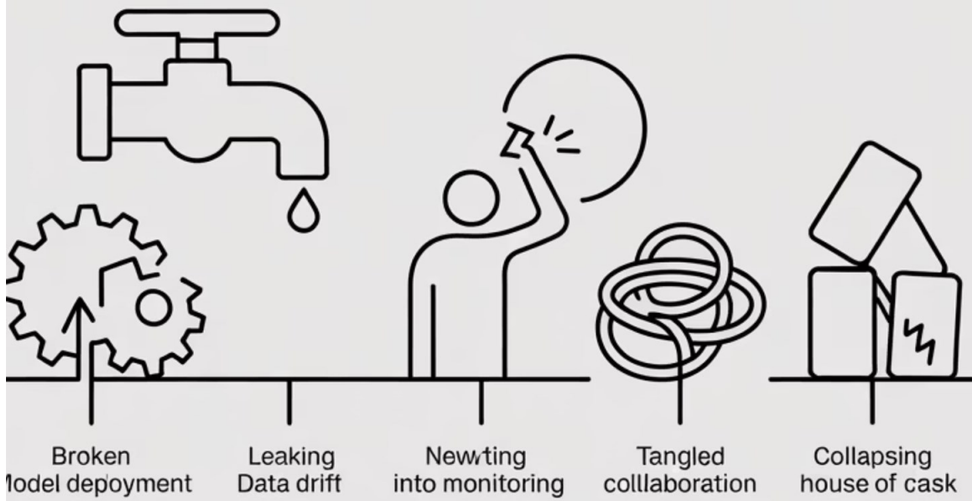
Проблемы без ModelOps

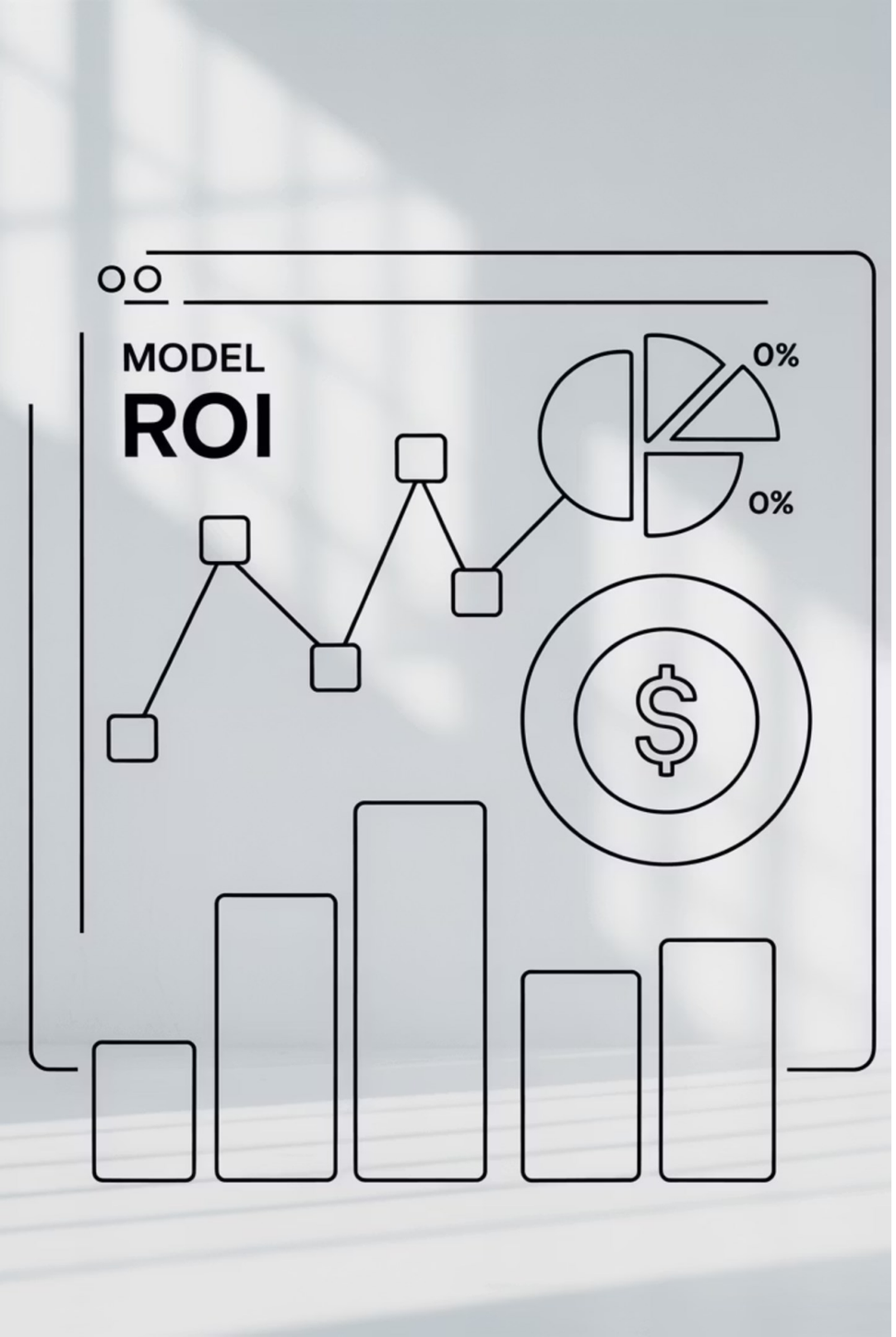
> «Модель работает, но никто не знает, как»

> Невозможно повторить результат

> Нарушение регуляторных требований

> Доверие бизнеса = 0





ModelOps и бизнес KPI

Связь:

1. **Модель**
   ML алгоритм
2. **прогноз спроса**
   Предсказание
3. **оптимизация запасов**
   Бизнес-процесс
4. **снижение затрат**
   Результат

ModelOps позволяет измерить ROI модели

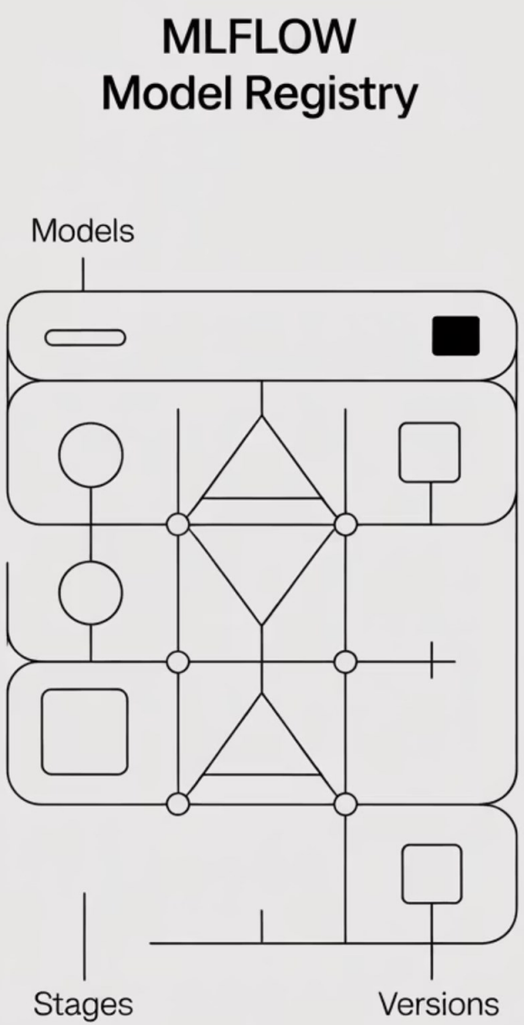
Дашборд: бизнес-метрики + технические метрики

Model Registry в действии (MLflow)

Скриншот MLflow UI:

- Версии
- Stage (Staging → Production)
- Переходы с комментариями

Переход в Production = только после валидации



Аудит и логирование

Что логируем:

- **Кто изменил статус модели**

  Пользователь и время

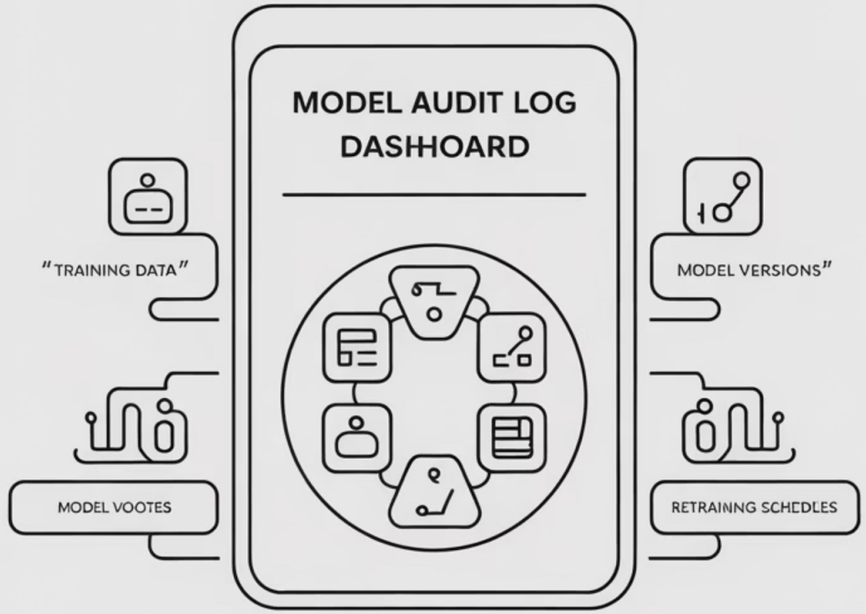
- **Когда запущен retrain**

  Расписание и триггеры

- **Какие данные использованы**

  Версии датасетов

Храним в PostgreSQL + Grafana для анализа





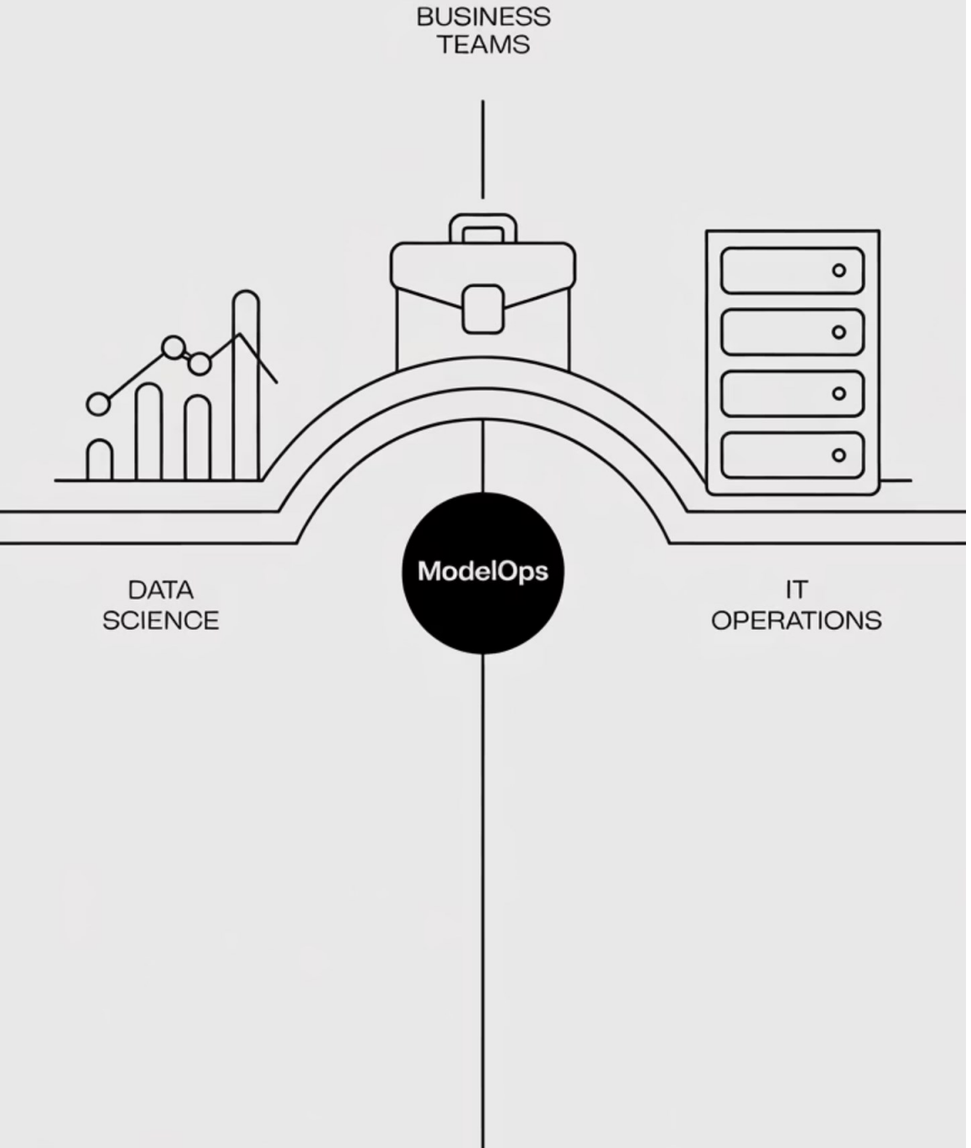
Архивация моделей

Почему архивируем:

- Юридические требования
- Возможность отката

Как:

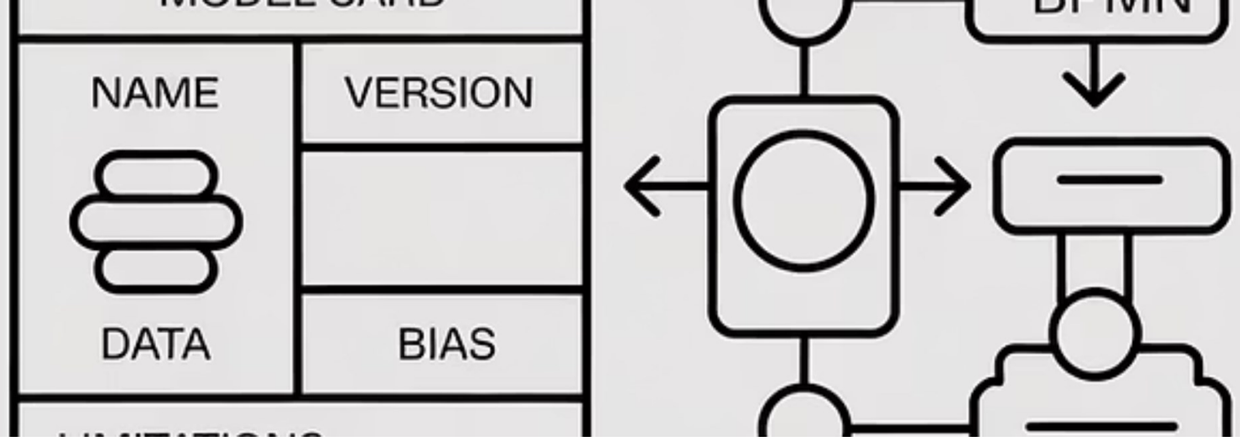
- Сохраняем Docker-образ
- Метаданные в MLflow
- Данные в DVC (frozen)



ModelOps это

Это мост между DS, IT и бизнесом

Это гарантия, что модель безопасна, объяснима и приносит ценность



Домашнее задание

Для вашего проекта:

01

Создать Model Card (Markdown)

Документация модели

02

Описать BPMN-процесс жизненного цикла модели в проекте

Workflow согласования